

ЛЕКЦІЯ 11

КОМП'ЮТЕРНО-МАТЕМАТИЧНЕ МОДЕЛЮВАННЯ ПРИРОДНИЧИХ ТА СОЦІАЛЬНО-ЕКОНОМІЧНИХ ПРОЦЕСІВ

1. КОМП'ЮТЕРНО-МАТЕМАТИЧНЕ МОДЕЛЮВАННЯ ПРИРОДНИЧИХ ПРОЦЕСІВ

2. КОМП'ЮТЕРНО-МАТЕМАТИЧНЕ МОДЕЛЮВАННЯ СОЦІАЛЬНО-ЕКОНОМІЧНИХ ПРОЦЕСІВ

1. КОМП'ЮТЕРНО-МАТЕМАТИЧНЕ МОДЕЛЮВАННЯ ПРИРОДНИЧИХ ПРОЦЕСІВ

Комп'ютерно-математичне моделювання природничих процесів – це міждисциплінарна галузь, яка використовує математичні методи та сучасні комп'ютерні технології для аналізу, опису та прогнозування різноманітних явищ у природі. Таке моделювання дозволяє створювати віртуальні моделі процесів, які складно або неможливо дослідити в реальних умовах.

Основні етапи комп'ютерно-математичного моделювання:

1. Формулювання задачі:

- Визначення основної мети дослідження.
- Вибір параметрів і змінних, що описують процес.

2. Математична постановка:

- Створення математичних моделей на основі фізичних законів, хімічних реакцій чи біологічних процесів.

- Використання диференціальних рівнянь, статистичних моделей, теорії ймовірностей, тощо.

3. Алгоритмізація:

- *Перетворення математичної моделі у форму, придатну для реалізації на комп'ютері.*
- Розробка алгоритмів для чисельного розв'язання рівнянь.

4. Реалізація моделі:

- Програмування моделі з використанням спеціалізованого програмного забезпечення (наприклад, **MATLAB, Python, COMSOL Multiphysics, ANSYS**).
- Вибір оптимальних чисельних методів.

5. Верифікація та валідація:

- Перевірка моделі на коректність (чи правильно вона відображає математичні принципи).
- *Порівняння результатів з експериментальними даними.*

6. Аналіз результатів:

- Інтерпретація отриманих даних.
- Візуалізація процесів (графіки, анімації, 3D-моделі).

7. Оптимізація:

- Удосконалення моделі для підвищення точності або швидкості розрахунків.

Приклади застосування:

- **Фізика:** моделювання кліматичних процесів, течій у рідинах, електромагнітних полів.
- **Хімія:** дослідження динаміки хімічних реакцій, розповсюдження домішок у середовищах.
- **Біологія:** симуляція популяцій, поширення епідемій, обмін речовин в організмах.
- **Геологія:** прогнозування землетрусів, вивчення динаміки магми, ерозійних процесів.

Переваги використання комп'ютерного моделювання:

- *Зменшення витрат на фізичні експерименти.*
- *Можливість тестування екстремальних сценаріїв.*
- *Висока швидкість обчислень для складних систем.*
- *Гнучкість у зміні параметрів та умов дослідження.*

ПРИКЛАД 1: моделювання кульової блискавки

[Ball lightning don't come closer]

Програмування моделі кульової блискавки — це складне завдання через *недостатньо вивчену природу явища*. Проте існують підходи, що дозволяють створити чисельну модель для вивчення основних характеристик кульової блискавки, зокрема її електромагнітної структури, температури та взаємодії з середовищем.

1. Постановка задачі

Ціль моделі — описати фізичні властивості кульової блискавки:

- форму (сферична структура),
- стабільність у просторі,
- енергетичний баланс,
- взаємодію з навколишнім середовищем (повітрям, твердими тілами).

2. Вибір математичної моделі

Для опису кульової блискавки можна використовувати:

- **Електродинамічні рівняння (рівняння Максвелла)** для моделювання електромагнітних полів.
- **Рівняння теплопередачі** для опису температурного розподілу.

- **Гідродинамічні рівняння** для моделювання взаємодії плазми з навколишнім середовищем.
- **Кінетичні рівняння плазми**, якщо вивчається поведінка заряджених частинок.

Фізичні основи моделювання кульової блискавки

Форма кульової блискавки визначається такими факторами:

- **Електромагнітні сили** — поля утримують заряджені частинки у сферичній формі.
- **Температура плазми** — визначає внутрішній тиск.
- **Гравітація та турбулентність** — впливають на стабільність структури.
- **Дифузія енергії** — з часом енергія розсіюється, що може змінювати форму блискавки.

Вибір математичної моделі

а. Електромагнітна модель:

Для опису електромагнітних полів, які впливають на форму, використовуються рівняння Максвелла:

$$\operatorname{div} \vec{D} = \rho$$

$$\operatorname{div} \vec{B} = 0$$

$$\operatorname{rot} \vec{E} = -\frac{\partial \vec{B}}{\partial t}$$

$$\operatorname{rot} \vec{H} = \vec{j} + \frac{\partial \vec{D}}{\partial t}$$

де:

$\vec{D}$ – вектор електричного зміщення;

$\vec{B}$ – вектор магнітної індукції;

$\vec{E}$ – вектор напруженості електричного поля;

$\vec{H}$ – вектор напруженості магнітного поля.

[CMM LEC 11-1]

```
import numpy as np
import matplotlib.pyplot as plt

# Параметри простору
x = np.linspace(-1, 1, 100)
y = np.linspace(-1, 1, 100)
X, Y = np.meshgrid(x, y)

# Початковий розподіл електричного поля
Ez = np.exp(-(X**2 + Y**2)/0.1))

# Еволюція полів
for t in range(100):
    Hz = np.gradient(Ez, axis=0) # Простий
    # розрахунок магнітного поля
    Ez = np.gradient(Hz, axis=1) # Оновлення
    # електричного поля

# Візуалізація
plt.contourf(X, Y, Ez, cmap='viridis')
plt.colorbar()
plt.show()
```

[CMM LEC 11-2]

```

import numpy as np
import matplotlib.pyplot as plt

# Просторові координати
r = np.linspace(0.1, 1, 100)
epsilon0 = 8.85e-12
q = 1e-6

# Електричне поле
E = q / (4 * np.pi * epsilon0 * r**2)

# Візуалізація
plt.plot(r, E, label='Електричне поле')
plt.xlabel('Радіус (м)')
plt.ylabel('Електричне поле (В/м)')
plt.title('Електричне поле кульової блискавки')
plt.grid()
plt.legend()
plt.show()

```

[CMM LEC 11-3]

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

# Створюємо порожнє полотно
fig, ax = plt.subplots()

# Налаштування осей
ax.set_xlim(-10, 10)
ax.set_ylim(-10, 10)
ax.set_aspect('equal')
ax.axis('off')

# Створення кульової блискавки
num_points = 300
x = np.random.randn(num_points)
y = np.random.randn(num_points)

```

```

sizes = np.random.rand(num_points) * 50 + 50 #
Розміри точок

# Малюємо точки кульової блискавки
scatter = ax.scatter(x, y, s=sizes, c='yellow',
alpha=0.6)

# Оновлення функції для анімації
def update(frame):
    scatter.set_offsets(np.column_stack([x +
np.random.randn(num_points) * 0.1, y +
np.random.randn(num_points) * 0.1]))
    scatter.set_sizes(np.random.rand(num_points) *
50 + 50)
    return scatter,

# Анімація кульової блискавки
ani = FuncAnimation(fig, update, frames=100,
interval=50, blit=True)

# Показуємо результат
plt.show()

```

ПРИКЛАД 2: моделювання і прогнозування землетрусів

Прогнозування землетрусів — це складний і багатоаспектний процес, спрямований на визначення ймовірності виникнення землетрусу в певному місці й у певний час. На сьогодні точне короткострокове прогнозування (з точністю до години або днів) залишається недосяжним через складність тектонічних процесів. Проте існують методи й підходи, які допомагають зменшити ризики й підготуватися до можливих катастроф.

Основні підходи до прогнозування

1. Геологічні дослідження

Дослідження тектонічних розломів, визначення зон сейсмічної активності та історичних даних про землетруси допомагають визначити найбільш небезпечні райони.

2. Сейсмічний моніторинг

Постійний моніторинг підземних поштовхів дозволяє виявляти мікросейсмічну активність, яка може передувати великому землетрусу.

3. Геофізичні методи

- **Зміни у магнітному полі Землі:** Аномалії магнітного поля можуть бути попереджувальними ознаками.
- **Деформація ґрунту:** Вимірювання зрушень і напруг у земній корі за допомогою GPS і інклінометрів.
- **Газовий аналіз:** Вивчення концентрації радону в ґрунтовому повітрі, оскільки її зростання може вказувати на сейсмічну активність.

4. Комп'ютерне моделювання

Аналіз історичних даних і моделювання динаміки тектонічних плит допомагає передбачити, де можуть виникати майбутні землетруси.

5. Тварини як "індикатори"

Є гіпотези, що деякі тварини реагують на фізичні чи хімічні зміни, які передують землетрусам. Проте цей метод залишається спірним і мало дослідженим.

Обмеження прогнозування

- *Землетруси часто виникають раптово, без явних попередніх ознак.*
- Розломи та зони тектонічного контакту мають складну структуру, що ускладнює точне прогнозування.

- Дані з прогнозів зазвичай використовуються для довгострокового планування та зменшення наслідків, а не для миттєвого попередження.

Технології раннього попередження

Системи раннього попередження (наприклад, в Японії, Мексиці та США) можуть виявляти первинні хвилі землетрусу (**P-хвилі**), які поширюються швидше за руйнівні **S-хвилі**. Це дає кілька секунд або хвилин для евакуації чи зупинки небезпечних процесів.

Майбутнє прогнозування

Сучасні наукові розробки з використанням штучного інтелекту, великих даних і високоточних геофізичних інструментів можуть значно покращити прогнозування. Зусилля з інтеграції глобальних даних і міждисциплінарних підходів також мають великий потенціал.

Моделювання динаміки тектонічних плит з використанням Python може бути виконано через обчислення напружень у земній корі, аналіз руху плит та використання геофізичних даних. Ось як це можна реалізувати:

Основні кроки:

1. Збір даних

- Використовуйте геофізичні дані, такі як історичні карти тектонічних плит, швидкість їхнього руху (дані GPS), і інформацію про зони розломів.
- Дані можна отримати з джерел, наприклад:
 - **IRIS Earthquake Browser**
 - **USGS Earthquake Catalog**

2. Обробка даних

- Використовуйте бібліотеки pandas для роботи з таблицями, numpy для числових обчислень, та matplotlib для візуалізації.

3. Моделювання руху плит

- Визначення векторів руху плит на основі GPS-даних.
- Симуляція розподілу напруження в зоні контакту плит. Це можна моделювати з допомогою методу скінченних елементів або спрощених моделей.

4. Прогнозування

- Створення моделі, яка аналізує накопичення напружень у зоні розломів і визначає ймовірні місця землетрусів.

[LEC 11-4]

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

# Параметри моделі
grid_size = 100 # Розмір сітки
time_steps = 200 # Кількість часових кроків
critical_stress = 100 # Критичне значення
напруження
stress_release = 50 # Напруження, яке
"вивільняється" при землетрусі

# Ініціалізація сітки напружень і швидкості руху
ПЛИТ
stress = np.zeros((grid_size, grid_size))
plate_velocity = np.zeros((grid_size, grid_size,
2))

# Задаємо швидкість руху плит (умовно – рух у x)
for i in range(grid_size):
    for j in range(grid_size):
        plate_velocity[i, j, 0] = 0.01 * (i -
grid_size // 2) # Рух у x
        plate_velocity[i, j, 1] = 0.01 * (j -
```

```

grid_size // 2)  # Рух у y

# Визначаємо зони розломів
fault_lines = [(50, 30), (50, 70)]  # Координати
розломів
for x, y in fault_lines:
    stress[x, y] = 90  # Початкове напруження

# Функція для оновлення напруження в часі
def update_stress(stress, velocity):
    new_stress = stress.copy()
    for i in range(1, grid_size-1):
        for j in range(1, grid_size-1):
            # Накопичення напруження від руху плит
            new_stress[i, j] += np.abs(velocity[i,
j]).sum()

            # Перевірка критичного напруження
            if new_stress[i, j] > critical_stress:
                new_stress[i, j] -= stress_release

# Розрядка
    return new_stress

# Створення анімації
fig, ax = plt.subplots()
img = ax.imshow(stress, cmap='hot',
interpolation='nearest', animated=True)
plt.colorbar(img, label="Напруження")

def animate(frame):
    global stress
    stress = update_stress(stress, plate_velocity)
    img.set_array(stress)
    return [img]

ani = FuncAnimation(fig, animate,
frames=time_steps, interval=100, blit=True)

plt.title("Еволюція напружень у часі")
plt.show()

```

2. КОМП'ЮТЕРНО-МАТЕМАТИЧНЕ МОДЕЛЮВАННЯ СОЦІАЛЬНО-ЕКОНОМІЧНИХ ПРОЦЕСІВ

Алгоритм Дейкстри

Існує багато способів знаходження мінімального шляху у графі. Точніше, оптимального маршруту. Під оптимальністю можна розуміти, наприклад, мінімальний час проходження маршруту, чи мінімальні витрати палива, витраченого на проходження маршруту, або мінімальну відстань. У цьому сенсі алгоритм Дейкстри досить ефективний та близький по суті і по принципу роботи до алгоритму Флойда. Але на відміну від останнього, дає змогу **знайти спектр найкоротших відстаней між вибраною вершиною графа та всіма іншими його вершинами**. Суть алгоритму Дейкстри полягає у порівнянні відстаней до сусідніх (інцидентних) вершин та знаходженні оптимального вибору на кожному кроці. Розглянемо рис. 12.1. Тут представлений зважений плоский простий неорієнтований граф. Вершини цього графа позначені цифрами. Біля ребер графа виставлені їх ваги. *Звернемо увагу, що ваги ребер зовсім не корелюють з їх геометричною довжиною, тому що їх визначення залежить від постановки конкретної задачі.* На цю обставину ми звернемо особливу увагу далі. А зараз лише зазначимо, що *вага ребра може являти собою будь-яку сутність*. Важливо зауважити, що *вага одного і того ж ребра часто буває змінною з часом величиною*. Розглянемо для прикладу транспортні магістралі між містами. У погожий літній день такі транспортні артерії володіють великою пропускну здатністю, і автомобілі ними рухаються з великими швидкостями. Але в зимовий період під час снігопадів та заметілей ситуація докорінно змінюється – транспортні засоби застрягають у снігових заметах, і тому прохідність такої дороги (тобто ребра графа) різко зменшується. Із цього випливає, що потрібно створити базу даних, у якій зберігаються ваги ребер та постійно оновлювати і базу даних, і ваги ребер відповідно до реалій. Наочно динамічність ваг ребер проявляється у випадку із транспортною мережею у місті, де ситуація змінюється щосекундно та з'являються затори на перехрестях.

Розглянемо граф (рис. 12.1). Нехай за допомогою алгоритму Дейкстри необхідно знайти найкоротші відстані між вершиною **1** та всіма іншими вершинами (рис. 12.1).

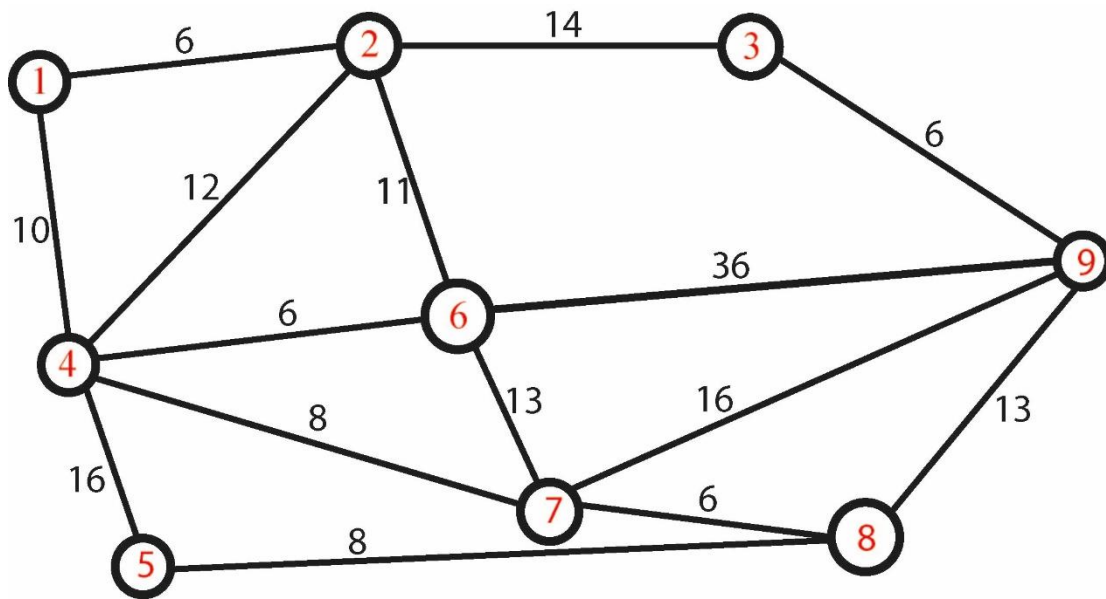


Рис. 12.1. Плоский неорієнтований зважений монограф, у якому за допомогою алгоритму Дейкстри знаходяться відстані між вершиною 1 та всіма іншими вершинами

Застосуємо програму (Лістинг 12.1) для обчислень найкоротших шляхів у цьому графі. Обчислювальне ядро алгоритму перебирає усі можливі варіанти вибору маршруту у графі та відшукує оптимальний. Далі наводимо усю програму повністю. Після виконання цієї програми достатньо лише ввести список суміжності для ваг ребер заданого графа, і програма обрахує оптимальний маршрут між вибраною вершиною (вершина 1) та всіма іншими вершинами графа.

[MiniWay-J]

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.util.ArrayList;
```

```

import java.util.Arrays;
import java.util.StringTokenizer;
public class MiniWay {
    private static int INF = Integer.MAX_VALUE/2;
    double weightU;
    int u;
    private int n; //кількість вершин у графі
    private int m; //кількість дуг у графі
    private ArrayList adj[]; //список суміжності
    private ArrayList weight[]; //вага ребра в орграфі
    private boolean used[]; //масив для зберігання інформації про пройдені
// та не пройдені вершини
    private double dist[]; //масив для зберігання відстані від стартової вершини
    private int[] pred; //масив предків, необхідних для відновлення
// найкоротшого шляху від стартової вершини
    int start; //стартова вершина, від якої знаходимо відстань до всіх інших
    private BufferedReader cin;
    private PrintWriter cout;
    private StringTokenizer tokenizer;
    private void dejkstra(int s) {//процедура запуску алгоритму Дейкстри зі
// стартової вершини
        dist[s] = 0; //найкоротша відстань до стартової вершини рівна 0
        for (int iter = 0; iter < n; ++iter) {
            int v = -1;
            double distV = INF;
            for (int i = 0; i < n; ++i) {
                if (used[i]) {
                    continue;
                }
                if (distV < dist[i]) {
                    continue;
                }
                v = i;
                distV = dist[i];
            }
            for (int i = 0; i < adj[v].size(); ++i) {
                int u = (int) adj[v].get(i);
                double weightU = (double) weight[v].get(i);
                if (dist[v] + weightU < dist[u]) {
                    dist[u] = dist[v] + weightU;
                    pred[u] = (int) v; } }
        }
    }
}

```

```

        used[v] = true;        }}
private void readData() throws IOException {
    cin = new BufferedReader(new InputStreamReader(System.in));
    cout = new PrintWriter(System.out);
    tokenizer = new StringTokenizer(cin.readLine());
    n = Integer.parseInt(tokenizer.nextToken());
    m = Integer.parseInt(tokenizer.nextToken());
    start = Integer.parseInt(tokenizer.nextToken()) - 1;
    adj = new ArrayList[n];
    for (int i = 0; i < n; ++i) {
        adj[i] = new ArrayList();
    }
    //ініціалізація списку, в якому зберігаються ваги ребер
    weight = new ArrayList[n];
    for (int i = 0; i < n; ++i) {
        weight[i] = new ArrayList();
    }
    //зчитуємо граф, заданий списком ребер
    for (int i = 0; i < m; ++i) {
        tokenizer = new StringTokenizer(cin.readLine());
        int u = Integer.parseInt(tokenizer.nextToken());
        int v = Integer.parseInt(tokenizer.nextToken());
        double w = Double.parseDouble(tokenizer.nextToken());
        u--;
        v--;
        adj[u].add(v);
        weight[u].add(w);
    }
    used = new boolean[n];
    Arrays.fill(used, false);
    pred = new int[n];
    Arrays.fill(pred, -1);
    dist = new double[n];
    Arrays.fill(dist, INF);
}

void printWay(int v) {
    if (v == -1) {
        return;
    }
    printWay(pred[v]);
    cout.print((v + 1) + " ");
}

```

```

}
private void printData() throws IOException {
    for (int v = 0; v < n; ++v) {
        if (dist[v] != INF) {
            cout.print(dist[v] + " ");
        } else {
            cout.print("-1 ");
        }
    }
    cout.println();
    for (int v = 0; v < n; ++v) {
        cout.print((v + 1) + ": ");
        if (dist[v] != INF) {
            printWay(v);
        }
        cout.println();
    }
    cin.close();
    cout.close();
}
private void run() throws IOException {
    readData();
    dejkstra(start);
    printData();
    cin.close();
    cout.close();
}
public static void main(String[] args) throws IOException {
    MiniWay solution = new MiniWay();
    solution.run();
}
}

```

Результат роботи програми:

```

9 15 1
1 2 6
1 4 10

```



```
4 2 12
4 6 6
4 7 8
4 5 16
5 8 8
7 8 6
7 9 16
8 9 13
6 7 13
6 9 36
6 2 11
2 3 14
3 9 6
0.0 6.0 20.0 10.0 26.0 16.0 18.0 24.0 26.0
1: 1
2: 1 2
3: 1 2 3
4: 1 4
5: 1 4 5
6: 1 4 6
7: 1 4 7
8: 1 4 7 8
9: 1 2 3 9
```

Process finished with exit code 0

Проаналізуємо програму Лістингу 12.1 та результат її роботи. Перша тріада чисел задає відповідно кількість вершин графа, кількість ребер графа та номер вершини, від якої розраховуються відстані до усіх інших вершин (у нашому випадку це вершина 1). Тепер з другого рядка у консолі вводимо тріади чисел у послідовності: номер однієї вершини, далі номер інцидентної вершини і третє число – вага відповідного ребра. І так вводимо всі 15 ребер графа (рис. 12.1) та натискаємо клавішу Enter. Після відпрацювання програма видає результат:

1) послідовність із 9 чисел, що і являють собою найкоротші відстані від вершини **1** до всіх інших вершин;

2) дев'ять рядків, що символізують собою оптимальний маршрут.

Скажімо, запис **9: 1 2 3 9** слід розуміти так – маршрут до вершини 9 пролягає вздовж ланцюга $1 \rightarrow 2 \rightarrow 3 \rightarrow 9$. Відповідно довжина цього оптимального ланцюга становить 26,0 одиниць.

Алгоритм Флойда

По суті, це досить простий алгоритм. Він дає змогу знайти спектр найкоротших відстаней між усіма вершинами графа. Принцип роботи алгоритму Флойда полягає у визначенні мінімальної відстані шляхом порівняння двох величин, а саме відстані між сусідніми вершинами графа, взятої напроям, та відстані між цими ж вершинами, але пройденої через інші два ребра, тобто обхідним шляхом. Ситуація наочно представлена на рис. 12.2.

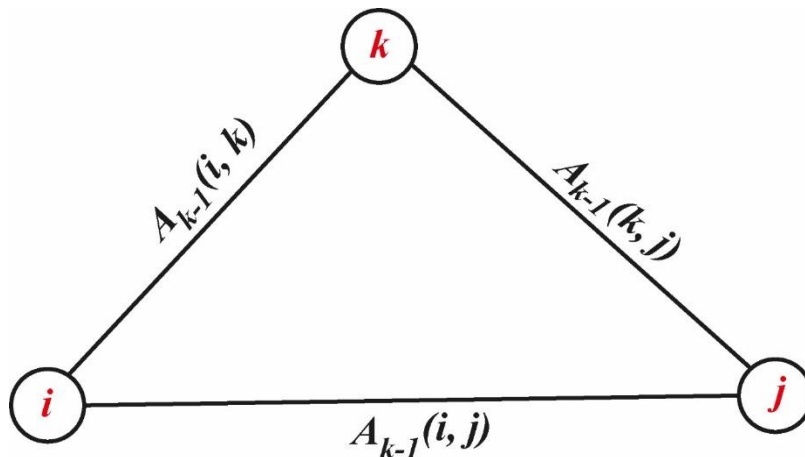


Рис. 12.2. Елементарний фрагмент у алгоритмі пошуку оптимального шляху у графі

Записати аналітично процедуру знаходження мінімальної відстані між сусідніми вершинами графа можна так:

$$A_k(i, j) = \min(A_{k-1}(i, j), A_{k-1}(i, k) + A_{k-1}(k, j)). \quad (1)$$

Графічно ситуація виглядає, як показано на рис. 12.2. Виникає питання: який обрати шлях від i до j – напроям чи застосувати обхідний маневр – пройти через вершину k ? Відповідь на поставлене питання дає формула (1) – із двох порівнюваних варіантів шляху треба вибрати коротший. Алгоритм перебирає всі можливі варіанти, тобто проходить по всіх вершинах k та знаходить найкоротший шлях між i та j . Далі описана процедура повторюється аж поки не буде знайдений оптимальний маршрут між усіма вершинами графа.

Зрозумілою тепер стає практична значимість графа, адже він дає можливість прокладати оптимальні маршрути, наприклад, між населеними пунктами області.

Нехай для графа, представленого на рис. 12.3, необхідно знайти мінімальні відстані між усіма вершинами. Для цієї мети використаємо алгоритм Флойда. Сформуємо для такого графа вагову матрицю A_{ij} відповідно до визначення (1).

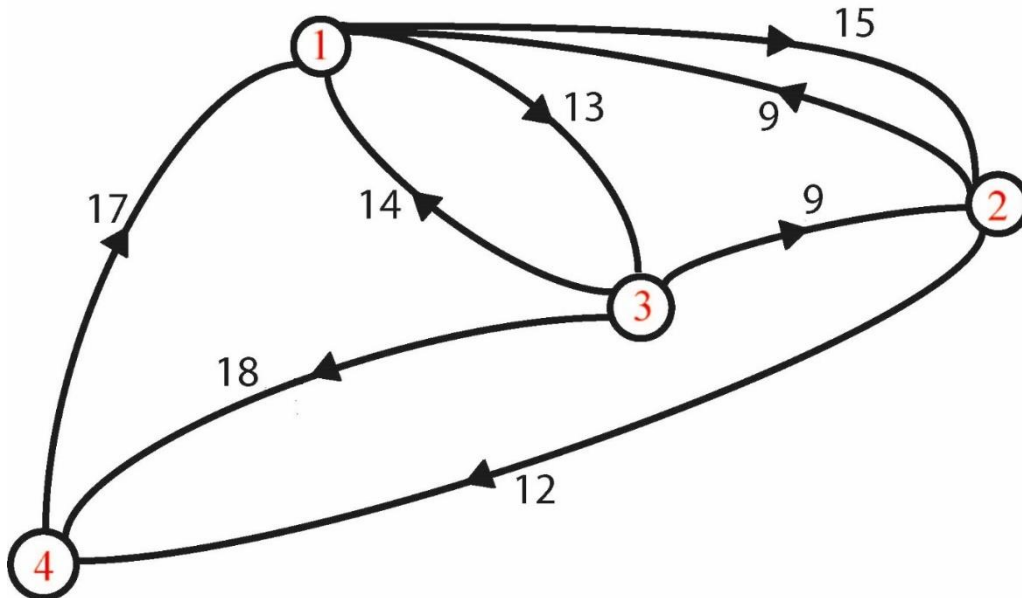


Рис. 12.3. Орієнтований навантажений планарний мультиграф

На нульовому кроці алгоритму це буде просто вагова матриця графа:

$$A_0 = \begin{pmatrix} 0 & 15 & 13 & \infty \\ 9 & 0 & \infty & 12 \\ 14 & 9 & 0 & 18 \\ 17 & \infty & \infty & 0 \end{pmatrix}.$$

На першому кроці ітераційного процесу зі співвідношення (1) отримаємо:

$$A_1(i, j) = \min (A_0(i, j), A_0(i, 1) + A_0(1, j)) . \quad (2)$$

Як бачимо, у цьому виразі змінна k приймає значення 1. Відповідна матриця матиме вигляд:

$$A_1 = \begin{pmatrix} 0 & 15 & 13 & \infty \\ 9 & 0 & 22 & 12 \\ 14 & 9 & 0 & 18 \\ 17 & 32 & 30 & 0 \end{pmatrix}.$$

Порівнюючи матрицю A_1 з матрицею A_0 , бачимо, що деякі елементи матриці A_1 змінились: внаслідок проходження по всіх елементах матриці з допомогою виразу (2) змінюємо ті елементи матриці, які виявляються меншими за попередні: наприклад, елемент $(A_1)_{2,3}$ став рівним 22, а в матриці A_0 цей елемент був рівним ∞ . Аналогічно проходимо другий, третій та четвертий ітераційні кроки, для яких k змінюється відповідно у напрямку $2 \rightarrow 4$ (у формулі (2) замість 1 виставляємо послідовно 2, 3 і 4). Остаточного матимемо матрицю:

$$A_4 = \begin{pmatrix} 0 & 15 & 13 & 27 \\ 9 & 0 & 22 & 12 \\ 14 & 9 & 0 & 18 \\ 17 & 32 & 30 & 0 \end{pmatrix}.$$

Власне, це і є матриця найкоротших відстаней. Наприклад, від вершини 1 до вершини 4 мінімальний шлях буде рівним 27 – це буде елемент матриці $(A_4)_{1,4}$. До речі, дістатися від вершини 1 до вершини 4 можна двома маршрутами: $1 \rightarrow 3 \rightarrow 4$ та $1 \rightarrow 2 \rightarrow 4$. Проте останній маршрут – коротший, тому алгоритм вибирає саме цей маршрут.

Отже, якщо A_0 – матриця ваг графа, то згідно з алгоритмом Флойда пробігаємо по всіх вершинах цього графа і шукаємо короткий шлях через вершину k , реалізуючи всі варіанти, що відповідають співвідношенню (1). Фактично для графа з V вершинами пройдено вище кроки зводяться до виконання потрібного циклу *for*:

```
{for (int k = 0; k < V; k++) //пробігаємо по всіх вершинах графа
for (int i = 0; i < V; i++)
for (int j = 0; j < V; j++)
if (dist[i][k] + dist[k][j] < dist[i][j]) {//знаходимо найкоротший шлях
dist[i][j] = dist[i][k] + dist[k][j];
next[i][j] = next[i][k];};
```

Звичайно, використовувати наведений вище формульно-словесний алгоритм Флойда для об'ємних графів нереально.

Для графа на рис. 12.4 (копія графа із рис 12.3), наведена знизу програма (Лістинг 12.2) дає змогу знаходити відстані за частки секунди (водночас як використання описового алгоритму в цьому випадку вимагатиме набагато більше часу).

Ваги ребер у Лістингу 12.2 задаються в основному коді цієї програми. Зверніть увагу, що ваги ребер у наведеному графі не корелюють зовсім із геометричними відстанями між вершинами ребер. У програмі Лістинг 12.2 двовимірний масив слід розуміти так: *перше число – це номер вершини, із якої виходить ребро, друге – це номер вершини, у яку входить ребро, і третє – це вага такого ребра*. І так задається будь-який граф. Запустимо програму та проаналізуємо результат її роботи:

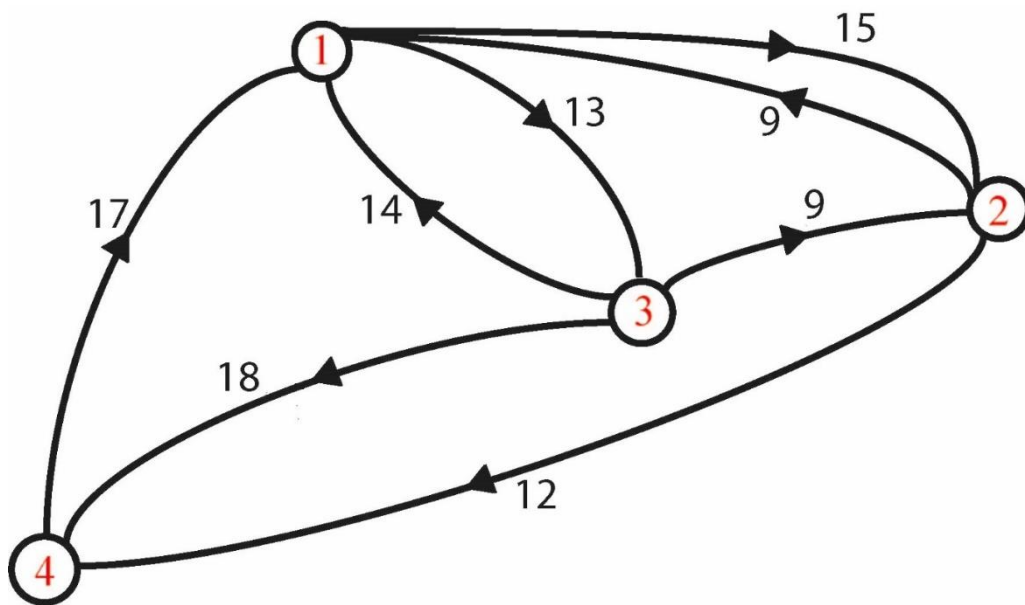


Рис. 12.4. Зважений орієнтований граф

[FloydWarshall-J]

Лістинг 12.2

```
import static java.lang.String.format;
import java.util.Arrays;
public class FloydWarshall {
    public static void main(String[] args) {
        int[][] weights = {{1, 2, 15}, {2, 1, 9}, {1, 3, 13}, {3, 1, 14},
                           {2, 4, 12}, {3, 4, 18}, {4, 1, 17}, {3, 2, 9}, {1, 4, 22}, {4, 2, 36}};
        };
        int numVertices = 4;
        floydWarshall(weights, numVertices);
    }
}
```

```

static void floydWarshall(int[][] weights, int numVertices) {
    double[][] dist = new double[numVertices][numVertices];
    for (double[] row : dist)
        Arrays.fill(row, Double.POSITIVE_INFINITY);
    for (int[] w : weights)
        dist[w[0] - 1][w[1] - 1] = w[2];
    int[][] next = new int[numVertices][numVertices];
    for (int i = 0; i < next.length; i++) {
        for (int j = 0; j < next.length; j++)
            if (i != j)
                next[i][j] = j + 1;
    }
    for (int k = 0; k < numVertices; k++)
        for (int i = 0; i < numVertices; i++)
            for (int j = 0; j < numVertices; j++)
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                    next[i][j] = next[i][k];
                }
    printResult(dist, next);
}

static void printResult(double[][] dist, int[][] next) {
    System.out.println("pair dist path");
    for (int i = 0; i < next.length; i++) {
        for (int j = 0; j < next.length; j++) {
            if (i != j) {
                int u = i + 1;
                int v = j + 1;
                String path = format("%d -> %d %2d %s", u, v,
                    (int) dist[i][j], u);
                do {
                    u = next[u - 1][v - 1];
                    path += " -> " + u;
                } while (u != v);
                System.out.println(path);
            }
        }
    }
}

```

Результат виконання програми:

Pair	dist	path
------	------	------

pair	dist	path
1 -> 2	15	1 -> 2
1 -> 3	13	1 -> 3
1 -> 4	22	1 -> 4
2 -> 1	9	2 -> 1
2 -> 3	22	2 -> 1 -> 3
2 -> 4	12	2 -> 4
3 -> 1	14	3 -> 1
3 -> 2	9	3 -> 2
3 -> 4	18	3 -> 4
4 -> 1	17	4 -> 1
4 -> 2	32	4 -> 1 -> 2
4 -> 3	30	4 -> 1 -> 3

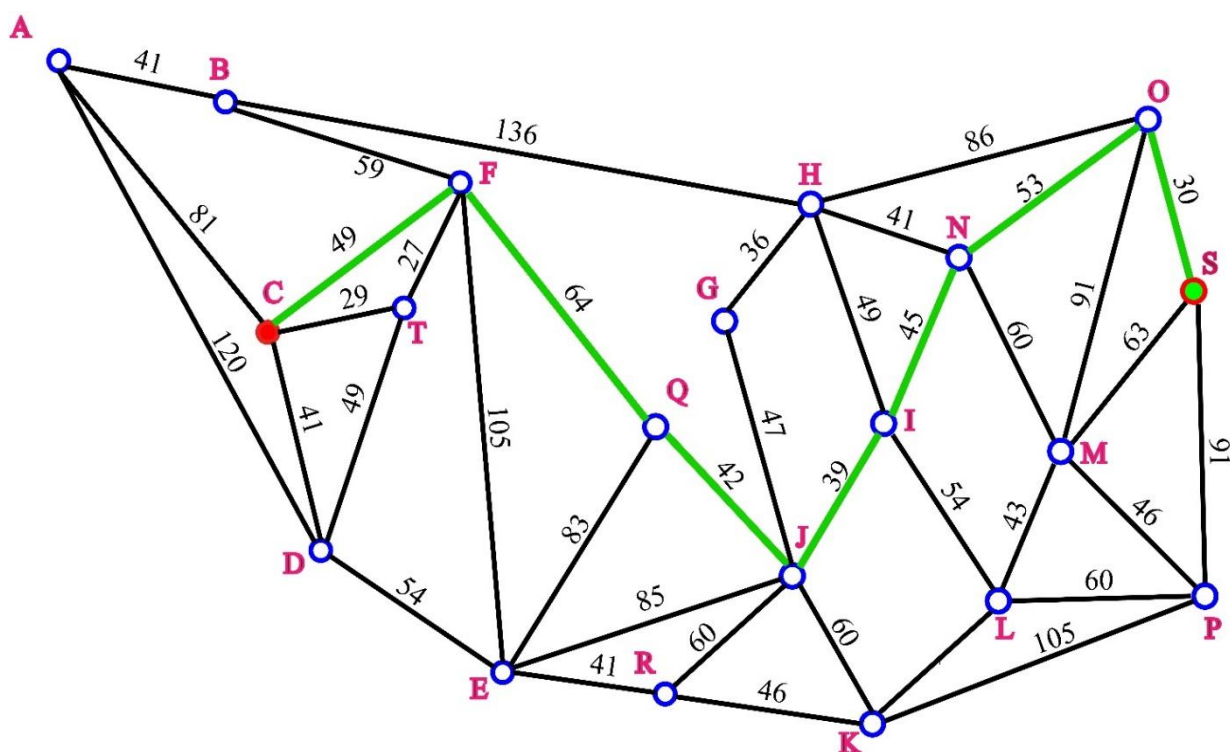
Process finished with exit code 0

Тут представлені найкоротші маршрути між усіма вершинами графа, а також довжини цих маршрутів. Скажімо, найкоротший шлях у напрямку від вершини 4 до вершини 3, тобто маршрут $4 \rightarrow 3$, дорівнює 30 та пролягає вздовж простого ланцюга $4 \rightarrow 1 \rightarrow 3$.

A-стар алгоритм

Нехай потрібно знайти оптимальний маршрут у мережі, представлений у вигляді графа на рис. 12.5. Почнемо розгляд поставленої задачі з практичної реалізації алгоритму, що знаходить найкоротший шлях у графі між двома вибраними вершинами. Це так званий A^* -алгоритм, або його ще називають евристичним. Особливість полягає у введенні поняття евристичної відстані. Позначимо цю відстань $h(x)$, де x – номер вибраної вершини графа. Це відстань, взята по прямій, від усіх вершин графа до вибраної кінцевої вершини. Далі вводиться функція $f(x) = g(x) + h(x)$, де $g(x)$ – відстань від стартової вершини до поточної вершини x , але пройдена вздовж ребер графа. Завдяки введенню евристики зменшується кількість можливих варіантів вибору шляху і тому

алгоритмічна складність A^* -алгоритму невелика. На відміну від алгоритмів Дейкстри та Флойда, цей алгоритм має лінійну алгоритмічну складність



(алгоритм Дейкстри – квадратичну, а алгоритм Флойда – кубічну).

Рис. 12.5. Зважений граф, у якому за допомогою A^* -алгоритму знаходиться оптимальний маршрут між вершинами С (стартова вершина) і S (фінішна вершина). Цей маршрут виділений потовщеною зеленою ламаною лінією. Біля кожного ребра проставлені ваги, що відповідають геометрії задачі

Представимо тепер приведений описовий варіант, а також програмну реалізацію A^* -алгоритму та покажемо шляхи його використання під час прокладання оптимальних маршрутів у різноманітних мережах, зокрема і міських транспортних. Розглянемо граф, представлений на рис. 12.5. Припустимо, треба знайти оптимальний маршрут між вершинами С та S. Скористаємось A^* -алгоритмом. Сформуємо таблицю, у яку спочатку занесемо згадані вже евристичні відстані $h(x)$. Фактично це особливі відстані, що вимірюються від поточної вершини x до кінцевої вершини маршруту по прямій. Отже, такі величини є найкоротшими з усіх можливих. Для чого вводиться евристична функція? Така процедура необхідна для знаходження мінімальних

величин функції $f(x)$ і на цій основі прокладання оптимального маршруту. Інакше кажучи, вибирається найкоротший ланцюг, що пролягає від стартової вершини через поточну вершину x до кінцевої. Для графа, представленого на рис. 12.5, оптимальний маршрут між вершинами С та S та його довжина прораховані та представлені у табл. 12.1.

Таблиця 12.1. Прокладання оптимального маршруту з допомогою A^* -алгоритму

Вершина	Відстань від С, $g(x)$	Евристична відстань, $h(x)$	$f(x)=g(x)+h(x)$	Попередня вершина
A	81,161	262	343,423	C, D
B		221	329	F
C	0	207	207	
D	41,78	208	249,285	C, T
E	154,196,96,240	201	355,397,297,441	F, Q, D, J
F	49,56	165	214,221	C, T
G	202	109	311	J,
H	243,280,238,378	86	329,366,324,464	I, N, G, O
I	194	79	273	J
J	155	117	272	Q
K	215,183	157	372,340	J, R
L	248	105	353	I
M	299,383	63	362,446	N, O
N	239	45	284	I
O	292	30	322	N
P		91		
Q	120	128	248	F
R	215, 137	176	391,313	J, E
S	322	0	322	O
T	29	178	207	C

Технічно процес прокладання найкращого шляху у павутинні графа виглядає так. На першому кроці визначаємо вершини, інцидентні до вершини С. Це будуть вершини А, F, Т та D. Записуємо відповідні дані $g(x)$ у табл. 12.1 у стовпчик для значень функції $g(x)$. Потім додаємо відповідні величини $g(x)$ та $h(x)$ і результат записуємо у стовпчик для функції $f(x)$. Порівнюємо між собою чотири отримані значення та вибираємо найменше число, рівне 214. Значить, наш маршрут має пролягати до вершини F. У вершини А, F, Т та D перехід відбувся із вершини С, тому у стовпчику «Попередня вершина» виставляємо С. Продовжуючи процедуру вибору оптимального маршруту, далі аналогічним способом заповнюємо табл. 12.1. Аналізуючи отримані результати, бачимо, що

протяжність оптимального маршруту $C \rightarrow S$ складає величину 322 умовні одиниці. З допомогою табл. 12.1 легко можна відтворити оптимальний маршрут у графі (рис. 12.5). Для цього зауважимо, що у вершину S перехід відбувся із вершини O . У вершину O перехід був здійснений із N . Продовжуючи подальший аналіз, відтворюємо сам оптимальний маршрут $C \rightarrow F \rightarrow Q \rightarrow J \rightarrow I \rightarrow N \rightarrow O \rightarrow S$.

Звернемо увагу, що у табл. 12.1 у стовпчику «Вершина» пройдені вершини зсовуються вправо. Це робиться для того, щоб під час оцінки функції $f(x)$ розглядати спектр величин $f(x)$ лише для непройдених вершин.

Представимо тепер програмний варіант сформульованого вище описового алгоритму, використовуючи програмний Java-код (Лістинг 12.3).

[A-STAR-(C-S)-J]

Лістинг 12.3

```
import java.util.PriorityQueue;
import java.util.HashSet;
import java.util.Set;
import java.util.List;
import java.util.Comparator;
import java.util.ArrayList;
import java.util.Collections;
public class AstarSearchAlgo {
    // евристична відстань h являє собою відстані по прямій між
    // всіма вершинами графа та фінішною вершиною S
    public static void main(String[] args) {
        // створення об'єктів – вузлів графа
        Node A = new Node("A", 262);
        Node B = new Node("B", 221);
        Node C = new Node("C", 207);
        Node D = new Node("D", 207);
        Node E = new Node("E", 201);
        Node F = new Node("F", 165);
        Node G = new Node("G", 109);
        Node H = new Node("H", 86);
        Node I = new Node("I", 79);
        Node J = new Node("J", 117);
        Node K = new Node("K", 157);
        Node L = new Node("L", 105);
        Node M = new Node("M", 63);
```

```
Node N = new Node("N", 45);
Node O = new Node("O", 30);
Node P = new Node("P", 91);
Node Q = new Node("Q", 128);
Node R = new Node("R", 176);
Node S = new Node("S", 0);
Node T = new Node("T", 178);
```

//створення об'єктів – ребер графа

```
A.adjacencies = new Edge[] {
    new Edge(B, 41),
    new Edge(C, 81),
    new Edge(D, 120), };
```

```
B.adjacencies = new Edge[] {
    new Edge(A, 41),
    new Edge(F, 59),
    new Edge(H, 136) };
```

```
C.adjacencies = new Edge[] {
    new Edge(A, 81),
    new Edge(F, 49),
    new Edge(T, 29),
    new Edge(D, 41), };
```

```
D.adjacencies = new Edge[] {
    new Edge(A, 120),
    new Edge(C, 41),
    new Edge(T, 49),
    new Edge(E, 54), };
```

```
E.adjacencies = new Edge[] {
    new Edge(D, 54),
    new Edge(F, 105),
    new Edge(Q, 83),
    new Edge(J, 85),
    new Edge(R, 41), };
```

```
F.adjacencies = new Edge[] {
    new Edge(B, 59),
    new Edge(Q, 64),
    new Edge(E, 105),
    new Edge(T, 27),
    new Edge(C, 49), };
```

```
G.adjacencies = new Edge[] {
    new Edge(H, 36),
    new Edge(J, 47), };
```

```
H.adjacencies = new Edge[]{  
    new Edge(B, 136),  
    new Edge(O, 86),  
    new Edge(N, 41),  
    new Edge(I, 49),  
    new Edge(G, 36) };  
I.adjacencies = new Edge[]{  
    new Edge(H, 49),  
    new Edge(N, 45),  
    new Edge(L, 54),  
    new Edge(J, 39), };  
J.adjacencies = new Edge[]{  
    new Edge(Q, 42),  
    new Edge(G, 47),  
    new Edge(I, 39),  
    new Edge(K, 60),  
    new Edge(R, 60),  
    new Edge(E, 85), };  
K.adjacencies = new Edge[]{  
    new Edge(R, 46),  
    new Edge(J, 60),  
    new Edge(L, 53),  
    new Edge(P, 105), };  
L.adjacencies = new Edge[]{  
    new Edge(K, 53),  
    new Edge(I, 54),  
    new Edge(M, 43),  
    new Edge(P, 60), };  
M.adjacencies = new Edge[]{  
    new Edge(N, 60),  
    new Edge(O, 91),  
    new Edge(S, 63),  
    new Edge(P, 46),  
    new Edge(L, 43), };  
N.adjacencies = new Edge[]{  
    new Edge(H, 41),  
    new Edge(O, 53),  
    new Edge(M, 60),  
    new Edge(I, 45), };  
O.adjacencies = new Edge[]{  
    new Edge(H, 86),
```

```

        new Edge(S, 30),
        new Edge(M, 91),
        new Edge(N, 53), };
P.adjacencies = new Edge[] {
    new Edge(K, 105),
    new Edge(L, 60),
    new Edge(M, 46),
    new Edge(S, 91), };
Q.adjacencies = new Edge[] {
    new Edge(F, 64),
    new Edge(J, 42),
    new Edge(E, 83), };
R.adjacencies = new Edge[] {
    new Edge(E, 41),
    new Edge(J, 60),
    new Edge(K, 46), };
S.adjacencies = new Edge[] {
    new Edge(O, 30),
    new Edge(P, 91),
    new Edge(M, 63), };
T.adjacencies = new Edge[] {
    new Edge(C, 29),
    new Edge(F, 27),
    new Edge(D, 49), };
AstarSearch(C, S);
List<Node> path = printPath(S);
System.out.println("Path: " + path);}
public static List<Node> printPath(Node target) {
    List<Node> path = new ArrayList<Node>();
    for (Node node = target; node != null; node = node.parent) {
        path.add(node);}
    Collections.reverse(path);
    return path;}
public static void AstarSearch(Node source, Node goal) {
    Set<Node> explored = new HashSet<Node>();
    PriorityQueue<Node> queue = new PriorityQueue<Node>(30,new
Comparator<Node>() {
    //реалізація методу порівняння
    public int compare(Node i, Node j) {
        if (i.f_scores > j.f_scores) {
            return 1;

```

```

    } else if (i.f_scores < j.f_scores) {
        return -1;
    } else {
        return 0;
    }
});

//вага на старті
source.g_scores = 0;
queue.add(source);
boolean found = false;
while ((!queue.isEmpty()) && (!found)) {
    //цей вузол має найнижчу величину f_score
    Node current = queue.poll();
    explored.add(current);
    //мета знайдена
    if (current.value.equals(goal.value)) {
        found = true;
    }
    //перевірка кожного інцидентного ребра поточного вузла
    for (Edge e : current.adjacencies) {
        Node child = e.target;
        double cost = e.cost;
        double temp_g_scores = current.g_scores + cost;
        double temp_f_scores = temp_g_scores + child.h_scores;
        //якщо інцидентний вузол оцінений і
        //оновлена f_score є більшою, то здійснюємо перехід
        if ((explored.contains(child)) &&
            (temp_f_scores >= child.f_scores)) {
            continue;
        }
        else if ((!queue.contains(child)) ||
            (temp_f_scores < child.f_scores)) {
            child.parent = current;
            child.g_scores = temp_g_scores;
            child.f_scores = temp_f_scores;
            if (queue.contains(child)) {
                queue.remove(child);
            }
            queue.add(child);
        }
    }
}

class Node {
    public final String value;
    public double g_scores;
    public final double h_scores;
    public double f_scores = 0;

```

```

public Edge[] adjacencies;
public Node parent;
public Node(String val, double hVal) {
    value = val;
    h_scores = hVal;}
public String toString() {
    return value;
}}
class Edge {
    public final double cost;
    public final Node target;
    public Edge(Node targetNode, double costVal) {
        target = targetNode;
        cost = costVal;
    }
}

```

Результат роботи програми виглядає так: Path: [C, F, Q, J, I, N, O, S] .

Важливість A^* -алгоритму полягає у тому, що відсікаються неперспективні варіанти маршрутів завдяки аналізу значень функції $f(x)$. Зі свого боку, величини $f(x)$ залежать від значень функції $h(x)$.

Практичні напрямки використання алгоритму Флойда

Алгоритм Флойда (або алгоритм Флойда-Варшалла) є ефективним методом для знаходження найкоротших шляхів між усіма парами вершин у зваженому графі. Його універсальність та простота реалізації роблять його корисним у багатьох практичних сферах. Ось основні напрямки його використання:

1. Оптимізація транспортних систем

- **Маршрутизація транспорту:** Визначення найкоротших шляхів для перевезення вантажів чи пасажирів між містами.
- **Планування логістики:** Розрахунок оптимальних маршрутів для зменшення витрат часу або палива.
- **Авіаційні маршрути:** Пошук найкоротших авіамаршрутів між аеропортами з урахуванням зупинок.

2. Інформаційні мережі

- **Маршрутизація в комп'ютерних мережах:** Використовується для аналізу та оптимізації передачі даних між вузлами мережі.
- **Моделювання затримок:** Обчислення мінімального часу передачі сигналу між мережевими пристроями.

3. Соціальні мережі та аналіз графів

- **Визначення "найближчих" зв'язків:** Пошук мінімальної відстані між користувачами або групами у соціальних мережах.
- **Аналіз впливу:** Використовується для оцінки центральності або важливості певного вузла у графі.

4. Географічні інформаційні системи (ГІС)

- **Пошук оптимальних маршрутів:** Побудова маршрутів на картах для транспортних засобів або людей.
- **Моделювання потоків:** Аналіз транспортних потоків у міських інфраструктурах.

5. Розв'язання задач в теорії ігор

- Використовується для аналізу матриць виграшів і розрахунку оптимальних стратегій.

6. Робототехніка та автоматизація

- **Маршрутизація роботів:** Планування оптимального переміщення роботів у складських приміщеннях або на виробничих лініях.
- **Уникнення зіткнень:** Розрахунок безпечних маршрутів для кількох роботів одночасно.

7. Біоінформатика

- **Порівняння геномів:** Використовується для обчислення мінімальних відстаней між елементами послідовностей ДНК чи білків.

- **Аналіз зв'язків у біологічних мережах:** Наприклад, у метаболічних шляхах або взаємодіях білків.

8. Ігрові додатки

- **Генерація ігрових карт:** Пошук найкоротших шляхів у світах із складною географією.
- **Штучний інтелект в іграх:** Планування дій персонажів у складних ігрових сценаріях.

9. Енергетичні системи

- **Оптимізація електромереж:** Розрахунок найкоротших шляхів передачі енергії між електростанціями і споживачами.
- **Зменшення втрат:** Аналіз мінімальних втрат енергії у мережах.

Алгоритм Флойда особливо корисний для графів середнього розміру, оскільки має складність $O(V^3)$, де V – кількість вершин. Для великих графів можуть бути ефективнішими інші алгоритми, як-от алгоритм Дейкстри або **Johnson's Algorithm**.